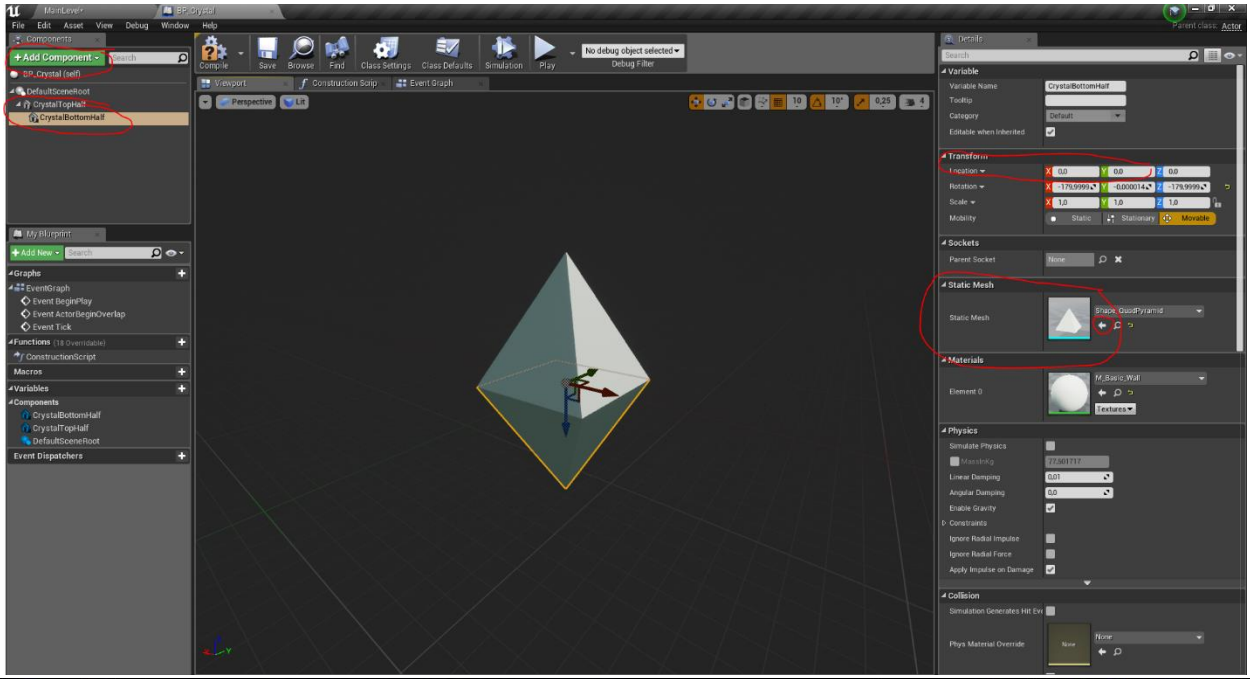
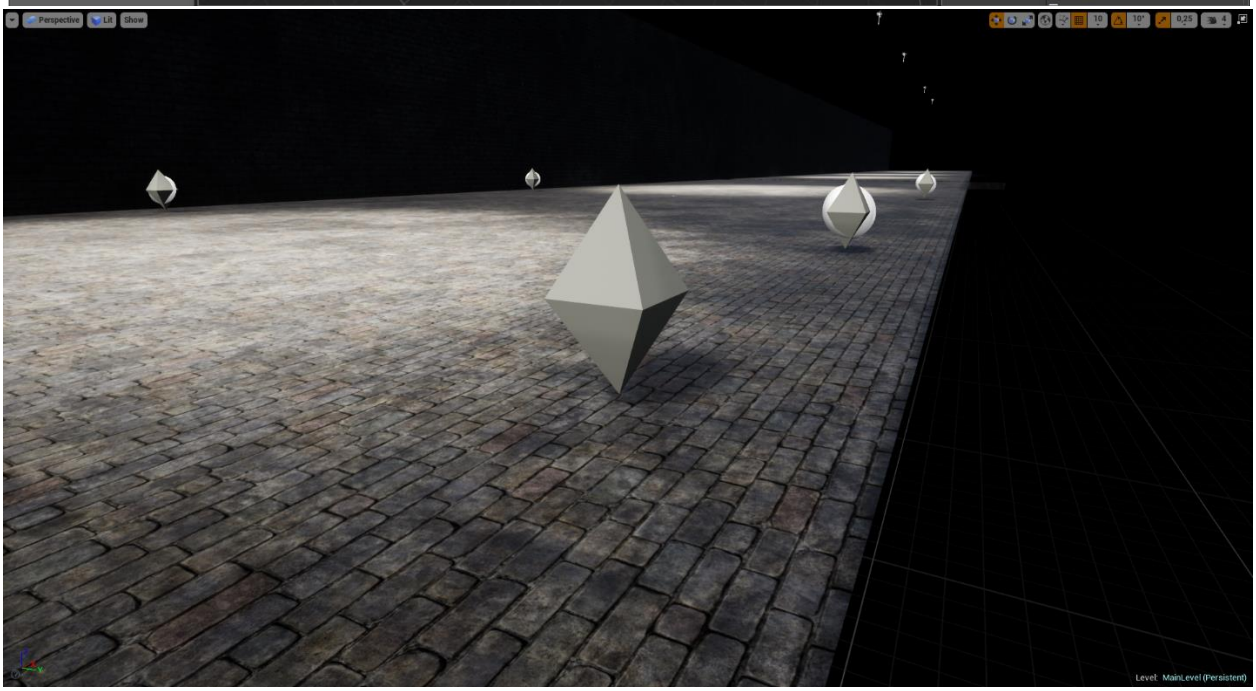
Klasy Blueprint - Actor

1. Wstępnie utworzyliśmy prosty poziom za pomocą pędzli BSP wraz z oświetleniem - w tej instrukcji posługując się klasą blueprintową Actor utworzymy obiekt (domyślnie będzie to świecący kryształ, ale możesz wymyślić własny obiekt), który gracz będzie mógł zbierać w czasie rozgrywki i dzięki temu będzie zdobywać punkty.
2. Dzięki klasom (w tym klasie Actor) możemy tworzyć obiekty, które mogą być wykorzystane w grze wielokrotnie, bez potrzeby tworzenia i konfigurowania ciągle każdego od nowa/ z osobna.
3. W silniku Unity takie obiekty nazywane są prefabami, z kolei w samym programowaniu (opartym na obiektach) również znajdziemy takie konstrukcje jak klasa i obiekt – klasa stanowi pewnego rodzaju formę, która posiada swoje właściwości a obiekty są już tylko „odlewami" stworzonymi na podstawie tej formy (przykładowo możemy mieć klasę o nazwie „Samochód" a obiektami będą już konkretne samochody takie jak: Audi A4 , BMW E46, FIAT PANDA itd.
4. Możemy więc w naszym przypadku postrzegać obiekt klasy Actor jako kontener, który przechowuje różnego rodzaju dokładnie skonfigurowane komponenty takie jak np. statyczną siatkę 3D, przełącznik, światło itd.
5. Dodatkowo taki obiekt klasy posiada grafy, czyli miejsce już nam dosyć dobrze znane bo właśnie w nim możemy zapisać naszą logikę/kod/node'y .
6. Wracając do UE4 ustalmy na początek miejsce, gdzie gracz powinien rozpocząć grę - możemy to zrobić wybierając z okna Modes -> Place ->Basic, obiekt o nazwie "Player Start".
7. Pierwszy odcinek/korytarz naszego poziomu powinien znajdować się na koordynatach xyz 0/0/0 a więc tam też najlepiej ustawić obiekt "Player Start".
8. Teraz przygotujmy sobie miejsce pod zapisywanie naszych klas bluprintowych - w folderze Content utwórzmy folder o nazwie "Blueprints".
9. Zanim utworzymy klasę blueprintową Actor należy zastanowić się co potrzebujemy, aby stworzyć w miarę estetycznie wyglądający kryształ stanowiący kopiowalny wzór/prefabrykant, który będzie wykorzystany wielokrotnie w naszej grze:
    - Potrzebny jest mesh/siatka – czyli wizualna reprezentacja naszego obiektu.
    - Potrzebny jest Trigger, który będzie odpowiedzialny za wykrywanie kolizji z graczem (tak aby gracz mógł zebrać kryształ).
    - Na pewno przydatne będą animacje – kryształy będą mogły obracać się w miejscu, a gdy zostanie wykryta kolizja z graczem to uruchomi się animacja „wciągania/zasysania/pochłaniania" kryształu.
    - Byłoby super gdyby kryształ emitował także jakieś własne światło.
10. Wewnątrz folderu "Blueprints" utwórzmy klasę bluprintową Actor – aby to zrobić należy kliknąć prawym przyciskiem myszy w okno Content Brower i wybrać opcje Blueprint Class.
11. Pojawi się okienko w którym możemy dokonać wyboru jakiego typu klasę chcemy utworzyć – w naszym przypadku zależy nam na klasie Actor. Przykładowa nazwa klasy to: BP_Crystal.
12. Kliknijmy dwukrotnie lewym przyciskiem myszy na utworzoną klasę.
13. Pojawiło się nam nowe okno, w którym znajdują się m.in takie sekcje jak Components i Viewport – w obu możemy dostrzec, że domyślnie utworzył się tam komponent o nazwie DefaultSceneRoot.
14. Utworzony komponent (biała sfera) to element który posiada zawsze każdy z obiektów klasy Actor znajdujących się w danej grze. Tę „białą kulkę" wraz z strzałkami możemy dostrzec w momencie gdy zaznaczmy jakiś obiekt tej klasy na scenie.
15. Gdy przesuwamy nasze obiekty i dokonujemy na nich także innych transformacji (obrotu, skalowania) to tak naprawdę wszystkie operacje skupiają się na tym domyślnie wygenerowanym komponencie i od niego są liczone.
16. Nasz „Root" (miejsce zerowe/rdzeń/korzeń/origin) to komponent do którego są podpięte wszelkie inne komponenty, które dodamy do naszej klasy – siatka 3D/triggery/ światło itd.
17. Czyli w momencie gdy wydawać się nam będzie że poruszamy meshem/siatką 3D/wizualną reprezentacją naszego kryształu, to tak naprawdę będziemy poruszać naszym originem/rdzeniem i na podstawie jego pozycji będą również obliczane pozycje wszelkich pozostałych komponentów.
18. Słowo „Root" jest szeroko wykorzystywane w branży technologicznej - przykładowo będzie ono kojarzone przez użytkowników różnego rodzaju dystrybucji linux lub jest ono także stosowane w systemie kontroli wersji git.
19. Podsumowując – nasz „korzeń" jest w hierarchii „rodzicem" dla wszystkich pozostałych komponentów.

20. Teraz po jakże wyczerpującym i nudnym wyjaśnieniu przejdźmy w oknie Components do przycisku „Add Component” i dodajmy statycznego mesha i spróbujmy mu samodzielnie nadać jakiś kształt za pomocą okienka Details i opcji Static Mesh
21. Przy dodawaniu przydatny może się okazać przycisk strzałki który znajduje się obok opcji Static Mesh, a także pomocny może okazać się folder o nazwie Shapes, który powinien znajdować się w folderze Starter Content – jeśli oczywiście chcemy korzystać z domyślnych modeli 3D.
22. Aby utworzyć prosty kształt kryształu to najlepszym modelem może się okazać model „piramidy” – po jej dodaniu będzie trzeba skopiować naszego mesha i odwrócić go o 180 stopni. Dzięki temu utworzy się nam prosty kształt przedstawiający kryształ.
23. Warto pamiętać o tym, aby:
    - Dodane komponenty siatki 3D znajdowały się na koordynatach 0/0/0.
    - Dolna część kryształu była w hierarchii „dzieckiem” dla górnej części – dzięki temu wszelkie transformacje będą wykonywane dla obu części naszego obiektu.
    - Stosować odpowiednie nazewnictwo do dodanych komponentów, aby w przyszłości się nie pogubić.
    - Elementy/kryształy które będzie zbierał gracz nie były za duże – warto tutaj poeksperymentować z skalą.
24. Efekt końcowy:





25. Proszę wykonać zrzut ekranu przedstawiający własną scenę wraz z nowoutworzonym obiektem klasy Actor.
26. [ZADANIE] Uzupełnij formularz Google: https://forms.gle/FL45uKqC6TwoJgLcA